

Air University



Digital Logic Design (Lab)

Year 2025

Ms. Faryal Naeem

Academic Year: 2024 – 2028

Department of Cyber Security

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 26, 2025

End Semester Project

Title: Simulation of a Basic Calculator Using Arduino

Objectives:

- To create a design and simulate a basic calculator by utilizing the Arduino library in the Proteus software environment.
- To execute fundamental arithmetic operations (addition, subtraction, multiplication, division) through a 4x4 matrix keypad input.
- To display both inputs and results on a 16x2 LCD within the Proteus simulation environment.
- To improve the understanding of embedded system design and interfacing concepts via virtual prototyping.

Equipment and Components:

- Proteus 8.11 software incorporating the Arduino library
- Arduino Uno module (within Proteus)
- 4x4 matrix keypad
- 16x2 LCD
- DC generator (5V supply)
- Connecting wires

Introduction:

This study focuses on simulating a simple calculator circuit in the Proteus environment, employing the Arduino library. By using virtual components such as the Arduino Uno, 4x4 keypad, and 16x2 LCD, this study eliminates the need for physical hardware. The Arduino is programmed by uploading a compiled code (.hex file) to mimic the behavior of the microcontroller. This calculator accepts numeric inputs and arithmetic operations through the keypad, processes these calculations in the Arduino simulation, and displays the results on the LCD screen. A DC generator supplies a stable 5V, ensuring that all components receive consistent power. This approach enables testing and debugging of embedded systems without relying on physical parts, thus lowering development time and costs.

Procedure:

A. Proteus Simulation Setup

1. Component Placement:

- Place the Arduino Uno module from the Proteus library onto the workspace.
- Add a 4x4 matrix keypad and a 16x2 LCD from the Proteus component library.
- Include a DC generator configured to supply 5V to power the Arduino, LCD, and keypad.

2. Wiring:

- Connect the keypad rows to digital pins 0, 1, 2, and 3 of the Arduino.
- Connect the keypad columns to digital pins 4, 5, 6, and 7 of the Arduino.
- Connect the pins RS, EN, D4, D5, D6, and D7 of the LCD to pins 8, 9, 10, 11, 12, and 13 of the Arduino, respectively.
- Ensure that all power, ground, and contrast connections for the LCD are made accurately according to the datasheet specifications.
- Connect the positive and negative terminals of the DC generator to the 5V and GND rails of the circuit, respectively.

3. Programming:

- Develop the Arduino code using the Arduino IDE.
- Compile the code and create the .hex file.
- Import the .hex file into the Arduino module within Proteus.

4. Simulation Execution:

- Start the simulation.
- Use the virtual keypad to input numbers and operations.
- Check the LCD for real-time displays of inputs, operations, and computed results.

B. Software Functionality

- The Arduino code monitors the keypad matrix to detect key presses.
- Numeric inputs are accumulated to form multi-digit values.
- The system recognizes and stores operators (A for addition, B for subtraction, C for multiplication, D for division).
- Pressing the # key activates the calculation, showing the result on the LCD.
- Pressing the * key clears the current input and resets the calculator.
- The LCD displays a custom division symbol and effectively manages division by zero situations.

Code Overview:

- Includes libraries for handling individual keys (Key.h), reading the keypad (Keypad.h), and controlling the LCD (LiquidCrystal.h).
- Defines the keypad as 4 rows and 4 columns.
- Sets up a 2D array keys that maps each button to its character (like '1', '2', 'A', etc.).
- Assigns Arduino pins 0–3 to keypad rows and pins 4–7 to columns.
- Creates a Keypad object with this setup to read key presses in the program.

```
7  #include <Key.h>
8  #include <Keypad.h>
9  #include <LiquidCrystal.h>
10
11  const byte ROWS = 4;
12  const byte COLS = 4;
13
14  char keys[ROWS][COLS] = {
15      {'7','8','9','D'},
16      {'4','5','6','C'},
17      {'1','2','3','B'},
18      {'*','0','#','A'}
19  };
20
21  byte rowPins[ROWS] = {0, 1, 2, 3};
22  byte colPins[COLS] = {4, 5, 6, 7};
23
24  Keypad kpd = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
25
```

- Sets Arduino pins 8–13 for the LCD.
- Creates an LCD object using those pins.
- Declares Num1 and Num2 for whole numbers.
- Declares Number for decimal results.
- Declares key and action for input and operator.
- result tracks if calculation is done.
- Defines a custom pattern for the division symbol.

```
26  const int rs = 8, en = 9, d4 = 10, d5 = 11, d6 = 12, d7 = 13;
27  LiquidCrystal lcd(rs, en, d4, d5, d6, d7);
28
29  long Num1, Num2;
30  float Number;
31  char key, action;
32  boolean result = false;
33
34  // Custom character for division symbol
35  byte division[8] = {
36      0x00, // Row 0
37      0x00, // Row 1
38      0x11, // Row 2
39      0x00, // Row 3
40      0x11, // Row 4
41      0x00, // Row 5
42      0x11, // Row 6
43      0x00, // Row 7
44  };
45
```

- Defines a function named `printNumber` that takes a decimal number (`num`) as input.
- Checks if `num` is a whole number (no decimal part) by comparing it to its floored value.
- Also checks if `num` fits within the range of a 32-bit signed integer.
- If both conditions are true, prints `num` as a long integer (without decimals) on the LCD.
- If not, prints `num` normally with decimals on the LCD.
- Ends the function.

```

46 // Function to print numbers appropriately
47 void printNumber(float num)
48 {
49     if (floor(num) == num && num <= 2147483647 && num >= -2147483648)
50     {
51         lcd.print((long)num);
52     }
53     else
54     {
55         lcd.print(num);
56     }
57 }
58

```

- `setup()` runs once when the Arduino starts.
- `lcd.begin(16, 2)` sets up a 16×2 LCD.
- `lcd.createChar(0, division)` makes a custom division symbol.
- `lcd.print("Calculator by ")` shows the title.
- `lcd.setCursor(0, 1)` moves to the second row.
- `lcd.print(" Project ")` shows " Project".
- `delay(800)` pauses for 0.8 seconds.
- `lcd.clear()` clears the screen.

```

59 void setup()
60 {
61     lcd.begin(16, 2);
62     lcd.createChar(0, division); // Create custom character fo
63     lcd.print("Calculator by ");
64     lcd.setCursor(0, 1);
65     lcd.print(" Project ");
66     delay(800);
67     lcd.clear();
68 }
69

```

- `loop()` runs again and again on the Arduino.
- It checks if a key is pressed on the keypad.
- If a key is pressed, it runs `DetectButtons()` to handle it.
- Then it updates the display with `DisplayResult()`.

```

70 void loop()
71 {
72     key = kpd.getKey();
73     if (key != NO_KEY) DetectButtons();
74     DisplayResult();
75 }
76

```

- `DetectButtons()` handles what happens when a key is pressed.
- `lcd.clear();` clears the LCD screen.
- If the * key is pressed (cancel):
- Prints "Button Cancel" to Serial Monitor.
- Resets all values and flags to start fresh.
- Then exits the function.

```

77 void DetectButtons()
78 {
79     lcd.clear();
80
81     // Handle cancel
82     if (key == '*')
83     {
84         Serial.println("Button Cancel");
85         Number = 0.0;
86         Num1 = 0;
87         Num2 = 0;
88         action = 0;
89         result = false;
90         return;
91     }
92

```

- Checks if the pressed key is a digit ('0' to '9').
- If a previous result was shown (`result == true`):
- It resets everything to start a new calculation.
- Sets `Number` to the digit pressed.
- If still entering a number:
- Adds the new digit to the current number (`Number = Number * 10 + digit`).
- Prints the digit to the Serial Monitor.
- `return;` ends the function after handling the digit.

```

93 // Handle digits
94 if (key >= '0' && key <= '9')
95 {
96     if (result == true)
97     {
98         // Start new calculation
99         Num1 = 0;
100        Num2 = 0;
101        action = 0;
102        result = false;
103        Number = key - '0'; // Start with this digit
104    }
105    else
106    {
107        // Append to current number
108        Number = Number * 10.0 + (key - '0');
109    }
110    Serial.println("Digit " + String(key));
111    return;
112 }
113

```

If an operator key (A, B, C, D) is pressed:

- Saves the current number as Num1.
- Clears Number for the next input.
- Sets the action based on the key:
- A = +, B = -, C = *, D = /.
- Resets the result flag to show a new operation has started.
- Prints the operator and exits the function.

```
114 // Handle operators
115 if (key == 'A' || key == 'B' || key == 'C' || key == 'D')
116 {
117     if (result == true)
118     {
119         // Use current Number as Num1 for chaining
120         Num1 = (long)Number;
121     }
122     else
123     {
124         // Normal case: set from current Number
125         Num1 = (long)Number;
126     }
127     Number = 0.0;
128     if (key == 'A') action = '+';
129     else if (key == 'B') action = '-';
130     else if (key == 'C') action = '*';
131     else if (key == 'D') action = '/';
132     result = false; // Reset result since new operation
133     Serial.println("Operator " + String(action));
134     delay(100);
135     return;
136 }
137
```

If # (equal key) is pressed:

- If an operator was chosen (action != 0):
- Saves the second number (Num2).
- Calls CalculateResult() to compute the answer.
- Sets result = true to show calculation is done.
- Prints "Button Equal".

Then exits the function.

```
138 // Handle equal
139 if (key == '#')
140 {
141     if (action != 0)
142     { // Only calculate if there is an action
143         Num2 = (long)Number;
144         CalculateResult();
145         result = true;
146         Serial.println("Button Equal");
147     }
148     return;
149 }
150 }
151
```

CalculateResult() performs the math based on the operator:

- + adds, - subtracts, * multiplies.
- / divides, but if Num2 is 0, sets Number to 0 to avoid error.
- Stores the result in Number.

```
152 void CalculateResult()
153 {
154     if (action == '+') Number = (float)Num1 + (float)Num2;
155     else if (action == '-') Number = (float)Num1 - (float)Num2;
156     else if (action == '*') Number = (float)Num1 * (float)Num2;
157     else if (action == '/')
158     {
159         if (Num2 == 0)
160         {
161             // Handle division by zero
162             Number = 0.0; // Could display an error
163         }
164         else
165         {
166             Number = (float)Num1 / (float)Num2;
167         }
168     }
169 }
170
```

- DisplayResult() shows data on the LCD.
- If no operator yet, it shows the first number.
- If an operator is selected:
- Shows Num1, the operator, and (if entered) the second number.
- Uses a custom symbol for division.

```

171 void DisplayResult()
172 {
173     lcd.setCursor(0, 0);
174     if (action == 0)
175     {
176         // Entering first number
177         printNumber(Number);
178     }
179     else if (!result)
180     {
181         // Operator selected, entering second number or just selected operator
182         lcd.print(Num1);
183         lcd.print(" ");
184         if (action == '/') lcd.write((byte)0); // Use custom character for division
185         else lcd.print(action);
186         if (Number != 0.0)
187         {
188             lcd.print(" ");
189             printNumber(Number);
190         }
191     }

```

- If the result is ready:
- Displays Num1, the operator, Num2, and the equals sign.
- Then shows the result (Number).
- Also prints the result on the second LCD row.

```

192 else
193 {
194     // Result calculated
195     lcd.print(Num1);
196     lcd.print(" ");
197     if (action == '/') lcd.write((byte)0); // Use custom character for division
198     else lcd.print(action);
199     lcd.print(" ");
200     lcd.print(Num2);
201     lcd.print(" = ");
202     printNumber(Number);
203 }
204 lcd.setCursor(0, 1);
205 printNumber(Number);
206 }

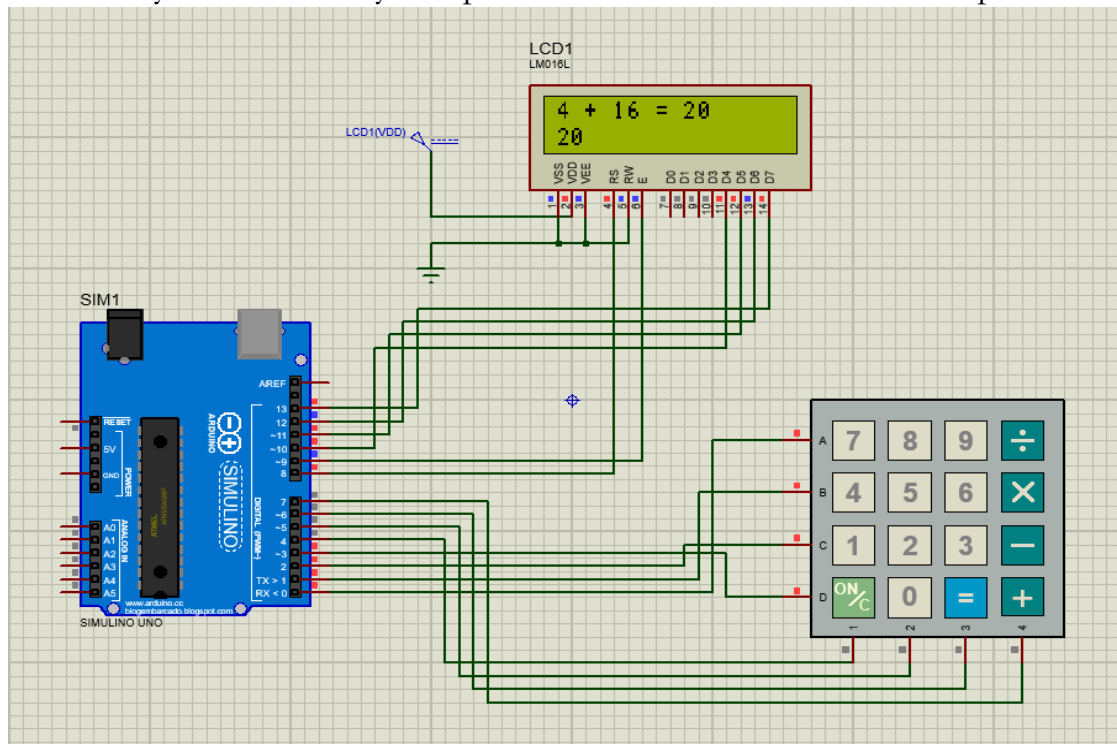
```

Observations:

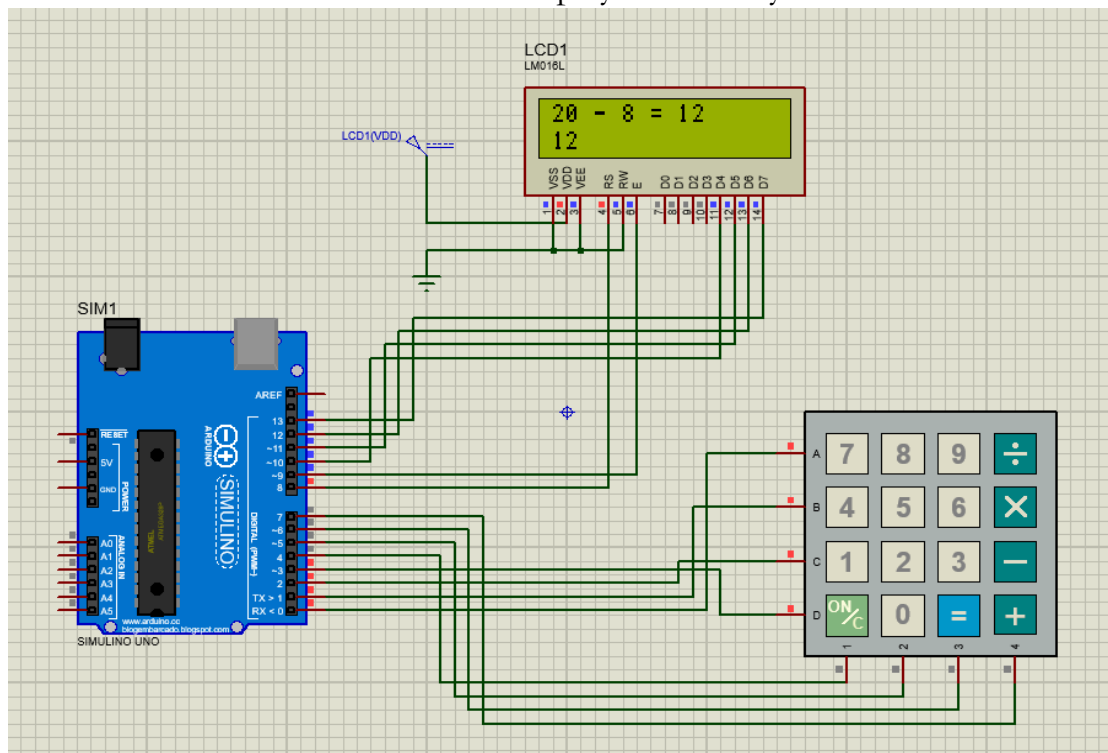
- The Proteus simulation effectively replicated the operations of the calculator.
- Keypad inputs were accurately detected and processed by the Arduino simulation.
- The LCD delivered clear and dynamic representations of inputs, operators, and results.
- All arithmetic operations were performed correctly, including proper handling of division by zero without errors.

Simulated Results:

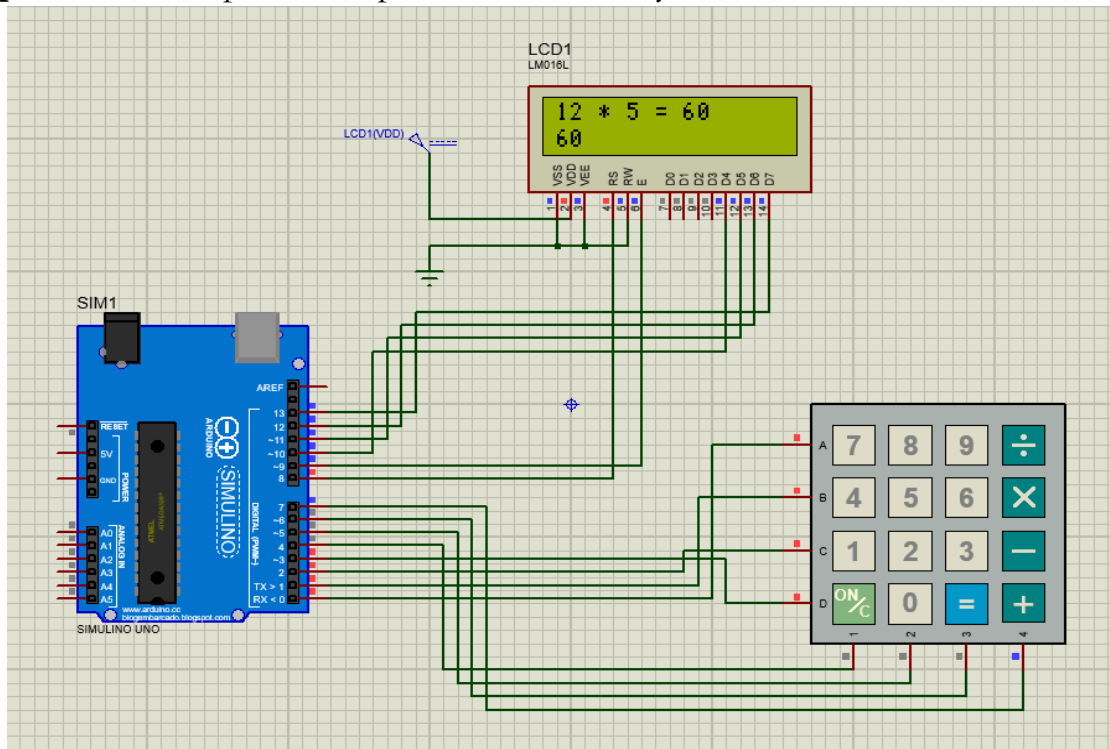
- **Addition:** The system accurately computes the sum of two numbers and presents it.



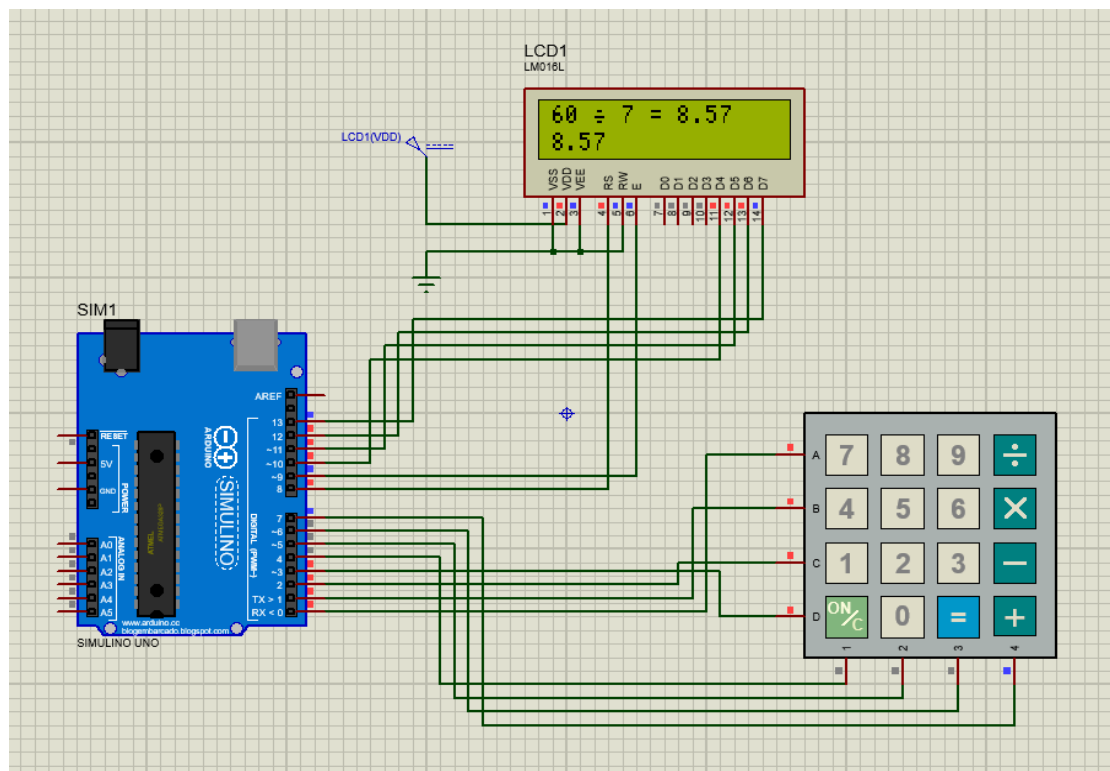
- **Subtraction:** The calculated difference is displayed correctly.



- **Multiplication:** The product is presented accurately.



- **Division:** The quotient is shown, or zero is indicated in cases where division by zero occurs.



Conclusions:

This experiment successfully illustrated the design and simulation of a basic calculator entirely within the Proteus environment using the Arduino library. The virtual setup afforded effective testing regarding keypad input processing, arithmetic calculations, and LCD output without requiring physical hardware. The integration of a DC generator supplying 5V ensured stable power throughout the simulation. The calculator executed all basic operations reliably, and the simulation verified the accuracy of both the code and circuit design. This approach exemplifies the benefits of virtual prototyping in embedded system development, facilitating rapid iterations and debugging prior to hardware deployment.

Authored By:

Amna Rashid

Muneeba Asjad

Khadija Baig

Rukhma Shehzadi

Muhammad Hamza

Asad Farooq

Zaeem Furqan

Usama Arshad

THE END
